

# gemstone-js vs gemstone-py

TypeScript and Python GemStone/S bridge comparison



Generated from the Markdown sources in [gemstone-rs/docs](#).

# Contents

gemstone-js vs gemstone-py

Quick Answer

Best Fit

Maturity Matrix

Where gemstone-js Is Strong

Where gemstone-py Is Still Ahead

Recommended Work Batches for gemstone-js

CLI Comparison

# gemstone-js vs gemstone-py

---

`gemstone-py` and `gemstone-js` solve the same broad problem from different runtime ecosystems. `gemstone-py` is the more mature Python path. `gemstone-js` is the async TypeScript path for JavaScript services, CLI tools, and npm-based delivery.

Use this guide when choosing between Python and TypeScript for GemStone/S application code. If the application already lives in Python, use `gemstone-py`. If the application already lives in Node, Deno, Bun, Express, Fastify, Fetch API, or Hono, `gemstone-js` is the natural fit once its native runtime path is proven for your platform.

## Quick Answer

| Question                                     | Better default           |
|----------------------------------------------|--------------------------|
| Production Python app or script              | <code>gemstone-py</code> |
| FastAPI, Litestar, or Django service         | <code>gemstone-py</code> |
| TypeScript service or CLI                    | <code>gemstone-js</code> |
| Express, Fastify, Fetch API, or Hono service | <code>gemstone-js</code> |
| Most mature visual database explorer today   | <code>gemstone-py</code> |
| npm package and TypeScript type workflow     | <code>gemstone-js</code> |
| Lowest operational risk today                | <code>gemstone-py</code> |

## Best Fit

| Use case    | <code>gemstone-py</code>                     | <code>gemstone-js</code>                                                    |
|-------------|----------------------------------------------|-----------------------------------------------------------------------------|
| Scripting   | Pythonic, low-friction scripts and notebooks | Possible, but async JavaScript is less convenient for small one-off scripts |
| Web apps    | FastAPI, Litestar, Django examples           | Express, Fastify, Fetch API, Hono adapters                                  |
| Async model | Python sync and async APIs                   | Async-first API throughout                                                  |

| Use case          | gemstone-py                                                    | gemstone-js                                                                              |
|-------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Package ecosystem | PyPI/TestPyPI and Python packaging                             | npm package shape and optional <code>@gemstone-js/native</code>                          |
| Type system       | Python type hints and generated access helpers                 | TypeScript signatures, manifests, decorators, and generated wrappers                     |
| Runtime support   | Python interpreters plus optional native acceleration          | Node native package, Deno/Bun FFI starters, mock runtime                                 |
| Visual tooling    | Python database explorer and VS Code workbench are more mature | Doctor, inspect, examples, benchmarks, migrations, but no equivalent visual explorer yet |

## Maturity Matrix

| Capability       | gemstone-py                                                                  | gemstone-js                                                                                                               |
|------------------|------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Install path     | Mature PyPI path                                                             | Alpha npm package shape, <code>0.1.0-alpha.0</code> locally                                                               |
| Native bridge    | Optional native acceleration already integrated into the Python release lane | Optional <code>@gemstone-js/native</code> , plus Deno/Bun FFI starters                                                    |
| Session API      | Mature sync and async session APIs                                           | Async-first <code>Session.connect()</code> , <code>execute()</code> , <code>perform()</code> , <code>performWith()</code> |
| Pooling          | Python web/session patterns                                                  | Session pool with reset-aware release, warmup, health checks, and lifecycle events                                        |
| Request scope    | FastAPI/Litestar/Django examples                                             | Shared <code>RequestScope</code> / <code>TransactionScope</code> across Express/Fastify/Fetch/Hono                        |
| Persistent roots | Supported                                                                    | <code>PersistentRoot</code> , <code>GsDict</code> , <code>OrderedCollection</code> , <code>GStore</code>                  |
| Migrations       | Supported in Python docs/examples                                            | Module-style migrations CLI and metadata roots                                                                            |
| Codegen          | Python codegen and typed access demos                                        | Manifest/decorator TypeScript codegen with schema validation                                                              |
| Benchmarks       |                                                                              |                                                                                                                           |

| Capability         | gemstone-py                                | gemstone-js                                                                               |
|--------------------|--------------------------------------------|-------------------------------------------------------------------------------------------|
|                    | Python performance docs and native checks  | Benchmark report/baseline/compare/register tooling                                        |
| Doctor/setup       | Python setup docs and verification scripts | <code>gemstone-js-doctor</code> for runtime, env, native import, and optional live checks |
| Explorer/workbench | Stronger today                             | Not yet equivalent                                                                        |
| Live CI confidence | Broader today                              | Needs more platform/runtime live coverage                                                 |

## Where gemstone-js Is Strong

- It is async-first, which matches modern JavaScript services.
- It has framework adapters for Express, Fastify, Fetch API, and Hono.
- It has a mock runtime, useful for TypeScript unit tests without a live stone.
- It has a rich package-tooling surface: doctor, examples, inspect, migrations, benchmarks, checksums, and API-contract checks.
- Its codegen model can use manifests, decorators, schemas, generated signatures, and TypeScript compile checks.
- It has high-level JavaScript value marshalling through `performWith()` and managed handles.

## Where gemstone-py Is Still Ahead

- `gemstone-py` is more mature as a published, documented user path.
- The Python database explorer and VS Code workbench are richer product surfaces today.
- Python examples cover more end-to-end workflows with lower setup friction.
- The Python release lane has more established PyPI/TestPyPI/native wheel/VSIX verification history.
- Live GemStone coverage is broader around sync, async, framework, native, and lifetime behavior.

## Recommended Work Batches for gemstone-js

| Batch | Work                                                                                        | Estimate    |
|-------|---------------------------------------------------------------------------------------------|-------------|
| 1     | Native publish confidence: npm package, @gemstone-js/native , clean install, live smoke     | 6-10 hours  |
| 2     | Visual tooling: explorer/workbench for inspect, browse, codegen, and persistent roots       | 10-18 hours |
| 3     | Installed examples: quickstart, web adapters, migrations, codegen, persistence helpers      | 5-8 hours   |
| 4     | Live CI: Node/Deno/Bun, framework adapters, pools, transactions, migrations                 | 8-14 hours  |
| 5     | Documentation/release polish: Medium article, PDF docs, release checklist, checksums        | 5-8 hours   |
| 6     | Cross-project alignment: shared concepts with gemstone-py and gemstone-rs explorers/codegen | 8-14 hours  |

Total: roughly **42-72 hours** to bring `gemstone-js` much closer to `gemstone-py` in maturity and product polish.

## CLI Comparison

From the `gemstone-rs` checkout:

```
gemstone-rs compare gemstone-js
gemstone-rs compare gemstone-js --gaps
gemstone-rs compare gemstone-js --next
gemstone-rs compare gemstone-js --totals
gemstone-rs compare gemstone-js --batches
gemstone-rs compare gemstone-js --json
gemstone-rs compare gemstone-js --gaps --json
gemstone-rs compare gemstone-js --next --json
gemstone-rs compare gemstone-js --totals --json
gemstone-rs compare gemstone-js --batches --json
gemstone-rs compare all --next
gemstone-rs compare all --totals
gemstone-rs compare all --next --json
```

Use the JSON form when an editor, release script, or dashboard needs to render the comparison. Use `compare all` when you want the TypeScript/Python and Rust/Python tracks in one report. `compare all --batches` includes the combined **11-batch, 67-117 hour** estimate for both tracks.

This PDF was generated locally from docs/. If you edit the Markdown sources, rerun `python docs/build_pdf_docs.py`.